花卉小能手

1．初识卷积神经网络

1.1 一张RGB三通道彩色图片是如何被处理成可用于训练的数据？张量又是什么？

首先一张图片可以用不同色彩模式存储，常用RGB色彩模式

RGB就是把一种颜色拆分为红，绿，蓝三种原色不同程度的混合，对于每一种原色，其强度的值用0~255的整数表示，从而每一种显示的颜色都可以用三个的数值表示，而把这三种颜色称为三个通道，即一张图可以被表示为一个三阶的张量（高，宽，3个色彩通道）

即：

1. 图像输入作为一个三维数组输入
2. 对数组的数值归一化，常见做法是除以255，放缩到[0,1]区间
3. 标准化，数值减去均值再除以方差

除此之外有的图像原始尺寸可能不符合模型输入，所以预处理会对原始图像拉伸，放缩等操作

张量原本是个数学/物理概念，在ml是指多维数组，即数组中的一个数据需要几个索引来唯一确定，就称为几维张量

常见的张量：

- 0维张量：标量
- 1维张量：向量
- 2维张量：矩阵
- 3维张量：如RGB图像
- 4维张量：一个批次的图像数据

1.2 一个卷积神经网络分为哪几层？

1. 输入层：接收预处理的图像数据
2. 卷积层：核心层，通过卷积运算提取图像局部特征
3. 激活层：通常紧跟在卷积层后，引入非线性函数对数据进行处理（如ReLU函数）
4. 池化层：下采样，减小特征图尺寸，降低计算量，防止过拟合
5. 全连接层：位于网络末端，将提取到的特征映射到样本标记空间，用于最终分类
6. 输出层：输出任务要的数据，分类通常使用Softmax函数输出各个类别的概率

1.3 什么是卷积，数学严格意义上的卷积和神经网络中的卷积操作有区别吗？卷积核、步长是什么？

在数学上，卷积是一种积分运算，描述两个函数生成第三个函数的数学算子，公式为

$$(f*g)(t)=\int f(\tau)g(t-\tau)d\tau$$

区别：数学上的卷积需要先翻转核函数再平移，而深度学习中的“卷积”实际上执行的是互相关操作，即不翻转核，直接在图像上滑动做点积，但在CNN语境下，习惯将其统称为卷积

卷积核：一个小的权重矩阵（如3x3或5x5），在图像上滑动，与对应位置的像素值进行点积运算，用于提取特定特征

步长：卷积核在图像上滑动的步幅，步长为1表示每次移动1个像素，步长为2表示每次移动2个像素

1.4 池化有哪些类型，其作用分别有什么？感受野是什么？

池化类型：

- **最大池化**：取窗口内的最大值（最常用，能保留最显著的特征）
- **平均池化**：取窗口内的平均值（保留整体背景特征）

作用：

1. 特征降维，减少计算量和参数数量
2. 扩大感受野
3. 在一定程度上提供平移不变性（图像稍微平移，池化后的结果变化不大）
4. **感受野**：卷积神经网络中某一层输出特征图上的一个像素点对应输入图像上的区域大小
   感受野越大，说明该神经元能“看到”的原图范围越广，越能捕捉全局信息

1.5 激活层在此的作用？最后卷积神经网络会输出什么？

激活层的作用：**引入非线性**

如果没有激活函数，无论神经网络有多少层，最终输出都是输入的线性组合

激活函数使得网络能够拟合非线性函数，从而具备强大的特征表达能力

最后输出什么：**取决于任务类型**

对于**图像分类任务**，最后通常经过全连接层和Softmax层，输出一个**概率分布向量**

向量的长度等于类别数，每个元素代表输入图像属于该类别的概率

2．数据预处理

2.1 数据集中存在异常或者损坏的图片无法打开

编写一个清洗脚本，在加载数据前遍历所有图片，尝试读取，如果抛出异常则将其删除或标记为损坏并移到一个特定的文件夹（代码详见仓库文件）

2.2 划分数据集

编写脚本按照一定比例完成数据集的划分，并创造不的文件

2.3 进行数据增强，扩充样本容量

对原始图片进行随机变换（如旋转、翻转、裁剪、变色），生成新的图片，从而增加样本多样性，提高模型鲁棒性(代码详见仓库文件)

2.4 调整图片大小以适应CNN的输入

数据预处理中，进行**缩放**，**填充**等方法

3．炼丹

3.1 什么是超参数？如何设定？

超参数是需要人为设定的参数，用于调整模型训练过程和模型结构

常见的超参数：

- 网络结构：层数、卷积核大小、卷积核数量、全连接层神经元数量
- 训练过程：学习率、批次大小、迭代轮数、dropout率等

如何设定

1. 参考经典项目在类似任务上的默认值
2. 设定几个候选值，穷举进行实验，找出效果最好的一组
3. 基于之前的实验结果，智能选择
4. 某些超参数可以采用动态的调整策略，如在训练过程中用余弦退火动态调整学习率

3.2 什么是优化器，有哪些类型?

定义：优化器用于更新模型参数以最小化损失函数的算法

常见类型：

- **SGD（随机梯度下降）**：最基础的优化器，每次仅使用一个或一小批样本计算梯度并更新
- **Momentum（动量法）**：在SGD基础上引入“动量”概念，模拟物理惯性，加速收敛并减少震荡
- **RMSprop**：调整学习率，对频繁更新的参数使用较小的学习率，对稀疏参数使用较大的学习率
- **Adam**：结合了Momentum和RMSprop的优点，能自适应调整每个参数的学习率，并引入类似·动量的概念
- **AdamW**：Adam的改进版，修正了Adam中权重衰减（L2正则化）的实现方式

3.3 DataLoader 与 DataSet 是什么及其使用?

DataSet（数据集类）：

一个**抽象类**，用于封装数据，告诉程序数据量以及如何获取一条具体的数据

DataLoader（数据加载器）：

对DataSet的包装，负责批量读取数据，并支持多进程加速、打乱顺序等操作

如何实现请看仓库代码

4. 代码设计思路

data_prepare有哪些部分：

1. 路径变量的统一管理
2. 基础配置和超参数
3. 几个主要函数的设定
4. 主流程

data_prepare要用的函数：

1. 扫描坏图函数
   将无法打开的图片路径写入bad（list）
2. 去除坏图函数
   用ImageFolder加载图片，根据bad去除打不开的图片路径，把剩余图片路径与标签以元组的形式保存进list（samples），返回类别list，samples，坏图数
3. 保存文件清单函数
   把samples写成对应路径的csv
4. 计算mean和std的函数
   对输入的samples，最终的两个list，分别是三通道的mean和std

后面才会用到的：

5. 图像变换函数
   输入mean和std，输出图像处理器train_tf,test_tf
6. 读清单文件函数
   输入每个图片的路径与类别的csv，输出装有图片路径和类别元组的list

7. 读预处理参数

   读data_prepare保存的json文件，输出json对象，顺便将json中的mean和std输入图像变换函数，得到两个图像变换器

5. 选做题

5.1 一直堆层数，能无限提高吗？模型层数越多、结构越深，性能就一定越好吗？

不是这样的

首先，深可能会有用：

1. **特征的层级组合更丰富**

   理想情况下，不同层可以做到学习不同的特征

   以本题的花卉识别为例：

   浅层学习边缘，中层学纹理特征，深层学整株花的形状

2. **感受野变大**

   层数越深，感受野也就越大，也就能够看到的特征越多，理论上的识别能力也会越强

3. **表达能力更强**

   层数越多，意味着能拟合的函数越复杂，特征的表达能力更强，对于复杂图像的识别理论上会更好

但是，深也会带来问题：

1. **梯度消失 / 爆炸**

   反向传播是**连乘**的链式法则，**每一层都会乘上一个小于 1 的数**（比如 sigmoid 的导数最大只有 0.25），

20 层之后，大概只有10^{-12}的水平，梯度极小，浅层的**变化慢**，**学不动**

2. **退化问题**

   一种当网络深度增加到一定程度后，训练误差不降反升的现象（**不是梯度消失，也不是过拟合**）

   退化问题的核心是：深层网络难以学习恒等映射

   一个更深层的网络在足够的时间里理论上不会比浅层的网络表现得更差（相同层参数保持与其一致，而更深的层保持恒等映射）

   然而，恒等映射需要权重精确等于单位矩阵，在非线性激活下几乎不可能（增加的每一层的权重在梯度下降中难以做到精确为1，往往会有一点偏差，这些偏差在一层一层的传递中累计，导致训练误差不降反升，拟合效果反而没有浅层模型好）

3. **数据量的约束**

   更深的网络需要更多的数据量才能发挥它更强的拟合能力，如果参数量远超模型，带来的很容易是过拟合，反而导致对于测试集的准确率下降

   行业经验：参数量大致应该和样本数在一个量级，或稍大

5.2 现代新的网络结构

1. **VGG**：

   核心思想：**只用 3×3 卷积，堆深**

   用小块卷积替代大块，减少参数

   参数集中在全连接层

   VGG16模型参考大小：1.38亿的参数量，527MB（量化后也要132MB）

2. ResNet18：

   核心思想：**残差连接**

   不让网络学"目标输出"，而让它学"**输出和输入的差**"（残差）

   普通： 目标 = H(x) ← 网络直接学 H

   残差： H(x) = F(x) + x ← 网络只学 F = H(x) - x，"跳跃连接"把 x 直接加

   如果最优解就是"什么都不改"（恒等映射），那么F(x) 只要学成 0 就行

   这比学成W = I容易得多，一定程度上解决了退化问题

加上残差连接后：50 层、101 层、甚至 1000 层以上都能正常训练，使得堆层数成为可行的策略

5.3 训练好的模型怎么用

项目已经实现了这件事，请看：

```
# 保存模型
def save_model(model):
    torch.save({"state_dict": model.state_dict()},MODEL_PATH)
```

用torch的save函数保存训练好的模型的参数，保存至特定路径的.pth文件，下次直接读取